

Uncertainty-Calibrated Diabetic Retinopathy Grading with Counterfactual Visual Explanations

Methodology, System Design and Architecture

Gayathri · Cohort E02 · Department of Computer Science and Engineering · 18 September 2026 · repository commit `ec6e161`

1. Problem statement and objectives

Diabetic retinopathy (DR) screening programmes photograph the retina and grade each image on the five-level International Clinical DR scale (0 = no DR ... 4 = proliferative DR). Deep networks now grade fundus photographs at specialist level, yet two properties block their use in screening: (i) their confidence scores are miscalibrated, so wrong answers look exactly like right ones; and (ii) their explanations are saliency heat-maps that show *where* the model looked but not *what would have to change* for the grade to differ, which is how clinicians reason about a borderline case.

Objectives

1. **Calibrated uncertainty.** Produce probabilities whose confidence matches empirical accuracy, and quantify predictive uncertainty per image with three post-hoc/ensemble methods.
2. **Selective prediction.** Convert uncertainty into a referral decision: grade the confident majority automatically and refer uncertain images to a clinician, measuring the exact accuracy-vs-workload trade-off.
3. **Counterfactual explanation.** Generate the minimal, realistic edit that moves an image to the adjacent grade and validate that the edited regions coincide with annotated lesions rather than noise.
4. **Deployability.** Export the calibrated grader to ONNX/TensorRT and quantify latency and memory on modest hardware.

Research questions

- **RQ1** Which uncertainty method (temperature scaling, MC dropout, deep ensemble) gives the best calibration (ECE, NLL) across CNN and transformer backbones, and does the ranking hold under distribution shift (APTOS → Messidor-2)?
- **RQ2** How much selective error is removed at 80 % and 90 % coverage, and which images get referred (image quality, grade)?
- **RQ3** Do latent-space counterfactuals localise to real lesions (IDRiD masks) better than Grad-CAM and a random-region baseline, and are they plausible (FID, blinded Likert)?
- **RQ4** Is explanation quality related to predictive uncertainty $u(x)$?
- **RQ5** What is the latency/memory cost of each uncertainty method at batch size 1 on CPU and on an FP16/INT8 GPU engine?

2. Methodology

2.1 Data and preprocessing

Four public, de-identified corpora are supported: APTOS 2019 (3 662 images, training/validation/test), EyePACS 2015 (35 126 images, optional pooled pre-training), Messidor-2 (1 744 images with adjudicated grades, held out entirely for distribution-shift evaluation) and IDRiD (516 graded images plus 81 images with pixel-level microaneurysm/haemorrhage/hard- and soft-exudate masks, used only to validate explanations). A synthetic corpus with the same schema (grade-dependent lesion blobs and masks) exercises the full pipeline without real data.

Splits are **patient-level** (both eyes of a patient in the same split), 70/15/15, stratified by the patient's worst grade, and frozen as CSV manifests. Preprocessing follows a fixed order: circular field-of-view crop from a green-channel threshold with padding to square $\rightarrow$ Graham local-contrast normalisation (subtract a Gaussian-blurred local mean, $\sigma = 10$ px at 512 px, gain 4) $\rightarrow$ bilinear resize to 512×512 $\rightarrow$ a heuristic quality score $q(x) \in [0,1]$ combining variance-of-Laplacian sharpness, illumination uniformity over an 8×8 grid and FOV coverage $\rightarrow$ ImageNet normalisation. Preprocessed images are cached to disk. Training augmentation (horizontal/vertical flips, $\pm 180^\circ$ rotation, $\pm 10\%$ scale, $\pm 15\%$ brightness/contrast) is declared in configuration; class imbalance is handled by a seeded, class-balanced sampler rather than duplicated oversampling.

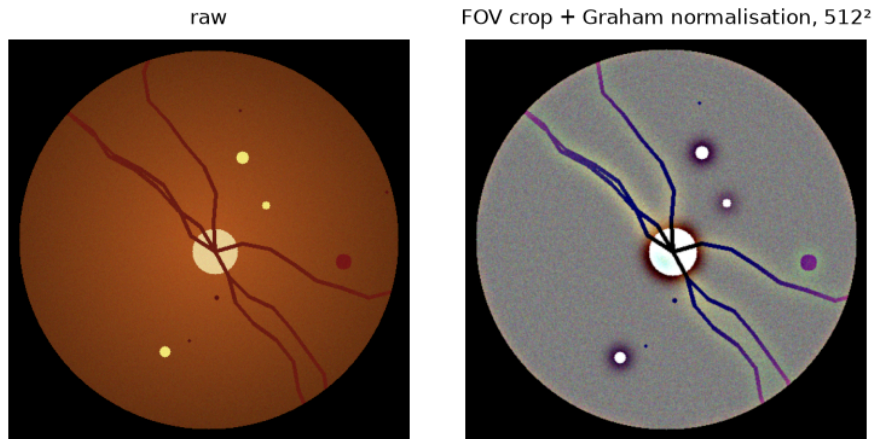


Fig. 1 — Preprocessing: raw synthetic fundus (left) and its cached FOV-cropped, Graham-normalised 512^2 version (right).

2.2 Grading model

Every backbone (ResNet-50, EfficientNet-B4, ViT-B/16 with gradient checkpointing, optional Swin-T) is created through timm with ImageNet weights and returns pooled features; a shared head applies dropout ($p = 0.2$) followed by a linear layer to five logits. Dropout is implemented functionally so that a single flag, `dropout_active`, keeps it stochastic at inference for Monte-Carlo sampling on all architectures. Training uses AdamW (3×10^{-4} head, 10^{-4} backbone, weight decay 0.01), a 3-epoch warm-up followed by cosine decay, cross-entropy with label smoothing 0.05, mixed precision, batch 16 at 512^2 , a maximum of 30 epochs and early stopping on validation quadratic-weighted kappa (QWK) with patience 6.

2.3 Uncertainty quantification

All methods implement one protocol, `fit(val_loader)` then `predict(x)  $\rightarrow$  (p, u)`, so the downstream decision layer is method-agnostic.

- **Temperature scaling.** A scalar $T > 0$ minimises validation NLL of $\text{softmax}(z/T)$, optimised in log-space with L-BFGS; the arg-max, and hence QWK, is unchanged.

- **MC dropout.** $T = 20$ stochastic passes; predictive mean $\bar{p}$ and either predictive entropy $H[\bar{p}]$ or mutual information $MI = H[\bar{p}] - \text{mean}_t H[p_t]$ (epistemic part).
- **Deep ensemble.** $M = 5$ independently seeded models; mean softmax; uncertainty as entropy of the mean or member disagreement (MI or variance).

Uncertainty scores share one implementation and one convention (higher = less confident): $1 - \max_k p_k$, entropy, mutual information. Calibration is measured with expected calibration error over 15 equal-width bins, maximum calibration error, NLL and multiclass Brier score; the in-house ECE is cross-checked against the netcal library in the test-suite.

$$ECE = \sum_b (n_b/N) \cdot |acc(b) - conf(b)|, \quad b = 1 \dots 15$$

2.4 Selective prediction

A gate accepts a prediction when $u(x) < \tau$ and otherwise emits REFER (−1). Sweeping τ over 200 quantiles of u on the *validation* split yields coverage and selective risk (error among accepted images); we report the area under the risk–coverage curve (AURC, computed exactly from the full ordering), the excess over the optimal AURC of a perfect ordering, and the selective error at 80 % and 90 % coverage. Thresholds chosen on validation are then applied to the test split, and a referral-budget table reports test accuracy/QWK when 5–50 % of cases are referred. Abstained cases are stratified by quality-score bin and by true grade to show *which* images are referred.

2.5 Counterfactual visual explanations

A StyleGAN2-ADA generator G is trained on the pooled training corpus (the code also ships a lightweight placeholder generator with the same interface). For an image x graded g , we (i) invert x to a latent w by minimising $\|G(w) - x\|^2 + \lambda \cdot \text{LPIPS}$, and (ii) optimise w' so that the frozen, temperature-scaled grader f assigns the adjacent grade $g' \in \{g - 1, g + 1\}$:

$$w' = \underset{w}{\operatorname{argmin}} CE(f(G(w))/T, g') + \lambda_1 \|G(w) - x\|_1 + \lambda_2 \text{LPIPS}(G(w), x)$$

The counterfactual is $x' = G(w')$ and the explanation map is $\Delta = |x' - x|$. Validation has three axes: (1) **decision flip** — does f actually predict g' on x' , and with what confidence; (2) **lesion consistency** on IDRiD — hit-rate and IoU between the top- k % of Δ and the MA/HE/EX/SE masks, against a random-region baseline of equal area and against Grad-CAM; (3) **plausibility** — FID between counterfactual and real fundus images plus a blinded 5-point Likert review by clinicians. Per-image uncertainty $u(x)$ is recorded with every explanation for the RQ4 analysis.

2.6 Deployment

The backbone and fitted temperature are exported as one ONNX graph (parity with PyTorch asserted at export), from which TensorRT FP16 and INT8 engines (entropy calibration on validation batches) are built. A profiler measures batch-1 latency (warm-up + N timed runs, median and p95) and peak memory for PyTorch FP32, ONNX Runtime CPU, TensorRT FP16/INT8 and the 20-pass MC-dropout path. A command-line tool returns a JSON decision for a single image.


```

class UQWrapper(Protocol):
    def fit(self, val_loader: DataLoader) -> None: ... # e.g. fit T; no-op otherwise
    def predict(self, x: Tensor) -> tuple[Tensor, Tensor]: ... # probs (B,5), uncertainty (B,)

class SelectiveGate:
    def __init__(self, tau: float, score: Literal["maxp", "entropy", "mi"]): ...
    def __call__(self, probs: Tensor, u: Tensor) -> Tensor: ... # grade 0..4, or -1 = REFER

@dataclass
class Counterfactual:
    x_prime: Tensor; delta: Tensor; target_grade: int; report: dict

def explain(x, g, g_prime, *, model, generator, cfg) -> Counterfactual: ...

```

3.2 Module design

Table 1 — Package decomposition.

package	modules	key public objects
dr_uq.data	loaders · splits · preprocess · quality · datamodule	FundusRecord, FundusDataModule, PreprocessCache, SeededWeightedSampler
dr_uq.models	backbones · grading_model	build_backbone, GradingModel, TemperatureScaled, LitGrader
dr_uq.uncertainty	scores · base · temp_scaling · mc_dropout · ensemble · factory	UQWrapper, NoUQ, TemperatureScaling, MCDropout, Ensemble, build_uq
dr_uq.selective	gate · risk_coverage · stratify	SelectiveGate, risk_coverage_curve, exact_auroc, optimal_auroc, stratify_abstentions
dr_uq.counterfactual	generator · invert · optimise · validate	PlaceholderGenerator, StyleGAN2Generator, invert, explain, lesion_consistency, grad_cam
dr_uq.evaluation	calibration · grading · runner · report	expected_calibration_error, grading_metrics, run_evaluation, aggregate
dr_uq.deploy	export_onnx · build_trt · profile · cli	export_onnx, OnnxGrader, build_engine, profile_backends, dr-uq CLI

3.3 Experiment design

Named experiment presets make each study a single command (Table 2). The main calibration study is a 3 backbones × 4 UQ methods × 3 seeds factorial; every evaluation writes per-image predictions so that paper tables are aggregated with 95 % bootstrap confidence intervals (1 000 image resamples, averaged over seeds). The distribution-shift study reuses the APTOS checkpoints and fits temperature and thresholds on APTOS validation only, avoiding any leakage from the external test corpus.

Table 2 — Experiment presets (configs/experiment/).

experiment	data	content	entry point
calib_sweep	APTOS	3 backbones × {none, temp. scaling, MC dropout, ensemble} × 3 seeds → calibration & selective-prediction tables	scripts/sweep.py
shift_messidor2	APTOS → Messidor-2	external evaluation of all APTOS models; T and τ from APTOS validation	scripts/sweep.py
cf_validation_idrid	IDRiD (81 masks)	counterfactuals for every masked image; lesion consistency vs random and Grad-CAM; FID; Likert template	scripts/explain.py

experiment	data	content	entry point
deploy_profile	—	ONNX export, TensorRT FP16/INT8, latency/memory profile	python -m dr_uq.deploy.*
synthetic_calib / smoke	synthetic	pipeline validation on the synthetic corpus (used for the preliminary results in this review)	scripts/sweep.py

3.4 Reproducibility design

- Global seeding of Python, NumPy, torch and CUDA; per-epoch seeded data-loader workers; cuDNN benchmark disabled for reported runs.
- Every run directory receives the fully resolved configuration, the git commit hash (with a dirty flag), the interpreter/torch versions and a `pip freeze`; Weights & Biases logging is optional and offline by default.
- Datasets are never downloaded automatically; loaders fail with acquisition instructions. Split manifests are deterministic for a seed and tracked as DVC stages.
- Notebooks are exploratory only; all reported numbers come from `scripts/evaluate.py` and `dr_uq/evaluation/report.py`.

Scope note

This is a research prototype built on public, anonymised corpora. It is not a medical device and does not diagnose patients.